A Course End Project Report on

Student Placement Tracking System using Python

A9511 - Python Programming Laboratory

Submitted by

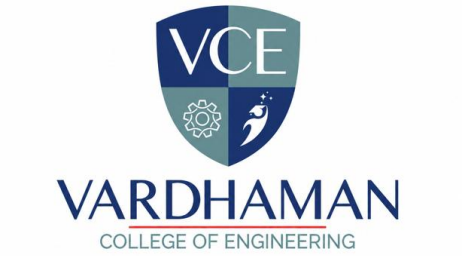
B. Sanjana

25881A0559

Course Facilitator

Dr. V. Lokeshwari Vinya

Associate Professor,CSE



Department of Computer Science and Engineering

Vardhaman College of Engineering

Hyderabad

2026-2027

CERTIFICATE

VARDHAMAN COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC** with **A**++ Grade
Kacharam, Shamshabad, Hyderabad- 501 218,
Telangana, India.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **" Student Placement Tracking System using Python"** is carried out by " **B. Sanjana** " with Roll Number " **25881A0559** " towards the A9511 - Python Programming Laboratory course in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering during the Academic year 2026-27.

Course Faculty
Dr. V. Lokeshwari Vinya
Assoc. Prof, CSE

Academic Head
Dr. R. Karthikeyan
Professor, CSE

Head of the Department
Prof. Y. Vijayalata
Department Of CSE

Contents

Abstract 4
1 Introduction 4
2 Literature Review 4
3 Objectives 5
4 Methodology 6
4.1 Architectural Diagram 6
4.2 Hardware and Software Requirements 6
4.3 Working Principle 7
4.4 Software Implementation 8
5 Results and Discussion 9
6 Mapping POs and SDGs 19
6.1 Program Outcomes (POs) Mapping 19
6.2 Sustainable Development Goals (SDGs) Mapping 19
7 Conclusion 20
Bibliography 21

Abstract

The project "Student Placement Tracking System using Python" focuses on designing and implementing an efficient system to manage and monitor campus placement activities for college students. The system allows coordinators to add student profiles, track academic eligibility (CGPA and backlogs), update placement offers, search records, and delete outdated entries. Each record contains attributes such as roll number, student name, branch, CGPA, number of backlogs, placement status, company name, and package offered.

Student records are managed in memory using built-in Python data structures—primarily a list of dictionaries with nested structures—and are permanently persisted using JSON file handling. Robust input validation and exception handling protect the system from invalid data entries and file input/output errors. This project bridges core data structures and real-world administrative automation in academic environments.

Keywords: Placement Tracking, Data Structures, Python, List, Dictionary, JSON, File Handling, CRUD Operations, Eligibility Filtering.

1 Introduction

Campus recruitment is a vital milestone for engineering students, connecting academic learning with professional careers. Managing student eligibility criteria (such as minimum CGPA thresholds and maximum allowable backlogs), tracking interview statuses, and recording placement offers manually or via scattered spreadsheets is inefficient and prone to discrepancies.

This project implements a console-based Student Placement Tracking System using core Python data structures (lists, dictionaries, tuples) and modular programming principles. It incorporates file handling via the json module to ensure data persistence across program executions. The system automates eligibility filtering, making it effortless for placement cells to shortlist eligible candidates for specific company drives.

2 Literature Review

Fundamental data structures like lists, dictionaries, and searching algorithms form the backbone of small-to-medium-scale record management applications. Traditional institutional workflows rely on complex enterprise databases, whereas lightweight administrative tools can be effectively prototyped using procedural and modular Python scripts.

While advanced enterprise portals deploy SQL relational databases and web frameworks, beginner-level implementations using built-in Python collections provide a solid foundation in

algorithmic thinking, time complexity evaluation, input validation, and structured file serialization.

3 Objectives

1. To design and implement a student placement tracking system using foundational Python data structures.
2. To implement automated eligibility checking based on dynamic criteria (CGPA thresholds and backlog limits).
3. To develop a menu-driven CRUD interface for managing student profiles and recruitment statuses (Add, View, Search, Update, Delete).
4. To validate user inputs rigorously and handle exceptions to ensure data integrity.
5. To ensure permanent data storage using JSON file handling.

4 Methodology

4.1 Architectural Diagram

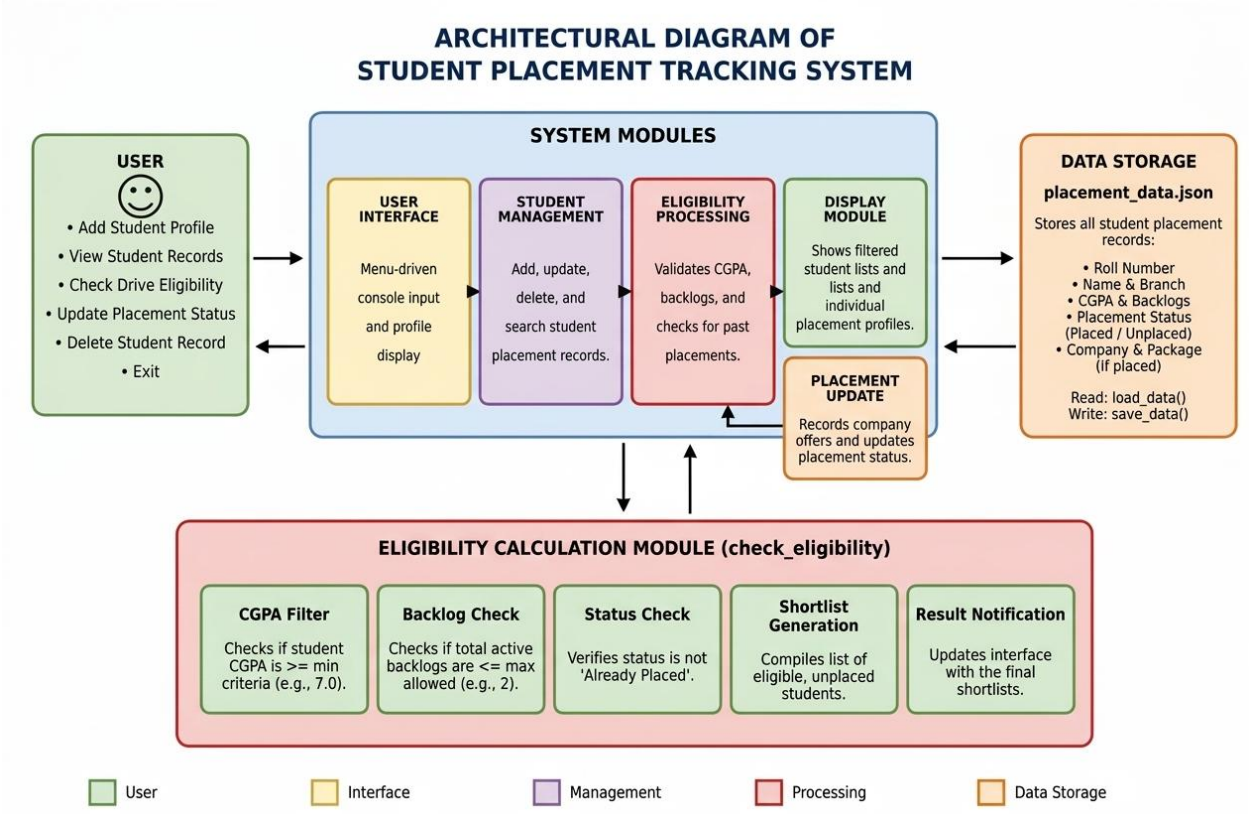


Figure 1: Architectural Diagram of Student Placement Tracking System

The system is divided into four main layers:

1. **User Interface Module:** A console-driven menu interface handling user inputs and command routing.
2. **Student & Placement Management Module:** Handles core operations including adding candidates, updating placement statuses, filtering eligible students, and deleting records.
3. **Eligibility & Processing Module:** Evaluates student credentials against company criteria (CGPA $\geq$ threshold and backlogs $\leq$ allowed limit).
4. **Data Storage Unit (placement_data.json):** Persistently saves and loads all candidate profiles using JSON serialization.

4.2 Hardware and Software Requirements

Hardware Requirements:

- Computer/Laptop with minimum 4GB RAM
- Intel i3 processor or higher
- Keyboard
- Monitor.

Software Requirements:

- Python 3.6 or higher, standard built-in libraries (json, os)
- Any standard IDE (VS Code, PyCharm, or IDLE).

4.3 Working Principle

1. **List (students):** Maintains the sequential collection of all student dictionaries.
2. **Dictionary (student):** Stores individual attributes (roll number, name, branch, CGPA, backlogs, status, company, package).
3. **Tuple:** Used for returning aggregated query results and status summaries.

Data Structures Used:

Data Structure	Where it is used	Purpose
Liststudents collection	**List**students collection	**List**students collection
Maintains records sequentially; supports easy iteration and removal.	Maintains records sequentially; supports easy iteration and removal.	Maintains records sequentially; supports easy iteration and removal.

Data Structure	Where it is used	Purpose
DictionaryIndividual student records	**Dictionary**Individual student records	**Dictionary**Individual student records
Maps attribute keys (e.g., 'CGPA', 'status') directly to values.	Maps attribute keys (e.g., 'CGPA', 'status') directly to values.	Maps attribute keys (e.g., 'cgpa', 'status') directly to values.
TupleFunction return valuesReturns combined eligibility or summary metrics safely.	**Tuple**Function return valuesReturns combined eligibility or summary metrics safely.	**Tuple**Function return valuesReturns combined eligibility or summary metrics safely.

4.4 Software Implementation

Algorithm Steps:

1. **Start** the program and load existing student records from placement_data.json (returns an empty list if the file doesn't exist).
2. **Display** the main menu options (Add Student, View All Records, Check Drive Eligibility, Update Placement Status, Delete Record, Exit).
3. **If Choice 1 (Add Student):** Read roll number, name, branch, CGPA, and backlogs; validate all inputs; check for duplicate roll numbers; save to file.
4. **If Choice 2 (View Records):** Iterate through the stored list and display all candidates in a formatted tabular layout.
5. **If Choice 3 (Check Eligibility):** Prompt for minimum CGPA and maximum backlogs; filter and display unplaced students meeting the criteria.
6. **If Choice 4 (Update Placement):** Search by roll number; update placement status, company name, and package offered; save changes.
7. **If Choice 5 (Delete Student):** Search by roll number; prompt for confirmation; remove the record from the list and save.
8. **If Choice 6 (Exit):** Terminate the execution loop.
9. **Repeat** until Exit is selected.

Pseudo Code:

```
START
LOAD list students FROM "placement_data.json"
WHILE true DO
  DISPLAY menu options (1 to 6)
  READ choice
  IF choice == 1 THEN
    CALL add_student(students)
  ELSE IF choice == 2 THEN
    CALL view_students(students)
  ELSE IF choice == 3 THEN
    CALL check_eligibility(students)
  ELSE IF choice == 4 THEN
    CALL update_placement(students)
  ELSE IF choice == 5 THEN
    CALL delete_student(students)
  ELSE IF choice == 6 THEN
    EXIT LOOP
  ELSE
    DISPLAY "Invalid choice error message"
```

```
END WHILE
END
```

5 Results and Discussion

Implementation Overview:

The system successfully manages student marks and calculates results automatically. The program is implemented using Python and demonstrates effective use of lists, dictionaries, tuples, and JSON file handling.

CODE:

```
import json
import os

DATA_FILE = "placement_data.json"
BRANCHES = ["CSE", "ECE", "EEE", "IT", "MECH", "CIVIL"]

# Load student records from JSON file
def load_data():
    if not os.path.exists(DATA_FILE):
        return []
    try:
        with open(DATA_FILE, "r") as file:
            return json.load(file)
    except (json.JSONDecodeError, OSError):
        print("Warning: Could not read placement data file.")
        return []

# Save student records to JSON file
def save_data(students):
    try:
        with open(DATA_FILE, "w") as file:
            json.dump(students, file, indent=4)
    except OSError:
        print("Error: Unable to save records to file.")

# Validate floating-point inputs (e.g., CGPA)
def get_float(prompt, minimum, maximum):
    while True:
        try:
            value = float(input(prompt))
            if minimum <= value <= maximum:
                return value
            print(f"Enter a value between {minimum} and {maximum}.")
        except ValueError:
            print("Invalid input. Please enter a valid number.")

# Validate integer inputs (e.g., Backlogs)
def get_integer(prompt, minimum):
    while True:
        try:
            value = int(input(prompt))
            if value >= minimum:
                return value
            print(f"Enter a value greater than or equal to {minimum}.")
```

```
        except ValueError:
            print("Invalid input. Please enter a whole number.")

# Display single student profile
def display_profile(student):
    print("\n" + "=" * 45)
    print("        STUDENT PLACEMENT PROFILE")
    print("=" * 45)
    print(f"Roll Number : {student['roll']}")
    print(f"Name        : {student['name']}")
    print(f"Branch      : {student['branch']}")
    print(f"CGPA        : {student['cgpa']:.2f}")
    print(f"Backlogs    : {student['backlogs']}")
    print(f"Status      : {student['status']}")
    print(f"Company     : {student['company']}")
    print(f"Package     : {student['package']}")
    print("=" * 45)

# Add new student record
def add_student(students):
    print("\n--- Add Student Profile ---")
    roll = input("Enter Roll Number: ").strip().upper()
    if not roll:
        print("Roll number cannot be empty.")
        return
    if any(s["roll"] == roll for s in students):
        print("A student with this roll number already exists.")
        return

    name = input("Enter Student Name: ").strip()
    if not name:
        print("Name cannot be empty.")
        return

    print("Available Branches:", ", ".join(BRANCHES))
    branch = input("Enter Branch: ").strip().upper()
    if branch not in BRANCHES:
        print("Invalid branch selected.")
        return

    cgpa = get_float("Enter CGPA (0.0 - 10.0): ", 0.0, 10.0)
    backlogs = get_integer("Enter Active Backlogs: ", 0)

    student = {
        "roll": roll,
        "name": name,
```

```
        "branch": branch,
        "cgpa": cgpa,
        "backlogs": backlogs,
        "status": "Unplaced",
        "company": "N/A",
        "package": "N/A"
    }

    students.append(student)
    save_data(students)
    print("Student profile added successfully!")
    display_profile(student)

# View all student records
def view_students(students):
    if not students:
        print("No student records found.")
        return

    print("\n--- All Student Placement Records ---")
    print(f"{'Roll':<12}{'Name':<18}{'Branch':<8}{'CGPA':<8}{'Status':<12}{'Company'}")
    print("-" * 70)
    for s in students:

print(f"{s['roll']:<12}{s['name'][:16]:<18}{s['branch']:<8}{s['cgpa']:<8.2f}{s['status']:<12}{s['company']}")

# Filter eligible students for a drive
def check_eligibility(students):
    print("\n--- Company Drive Eligibility Check ---")
    min_cgpa = get_float("Enter Minimum CGPA Required: ", 0.0, 10.0)
    max_backlogs = get_integer("Enter Maximum Allowed Backlogs: ", 0)

    eligible_students = [
        s for s in students
        if s["cgpa"] >= min_cgpa and s["backlogs"] <= max_backlogs and s["status"] == "Unplaced"
    ]

    if not eligible_students:
        print("No eligible unplaced students found for these criteria.")
        return

    print(f"\nEligible Candidates Found: {len(eligible_students)}")
    print(f"{'Roll':<12}{'Name':<18}{'Branch':<8}{'CGPA':<8}{'Backlogs'}")
    print("-" * 55)
    for s in eligible_students:
```

```
    print(f"{s['roll']:<12}{s['name'][:16]:<18}{s['branch']:<8}{s['cgpa']:<8.2f}{s['backlogs']}")

# Update placement status
def update_placement(students):
  roll = input("Enter Roll Number to update placement status: ").strip().upper()
  for s in students:
    if s["roll"] == roll:
      print(f"Current Status for {s['name']}: {s['status']}")
      company = input("Enter Placed Company Name: ").strip()
      package = input("Enter Offered Package (e.g., 6.5 LPA): ").strip()

      s["status"] = "Placed"
      s["company"] = company if company else "N/A"
      s["package"] = package if package else "N/A"

      save_data(students)
      print("Placement status updated successfully!")
      display_profile(s)
      return
  print("Student record not found.")

# Delete student record
def delete_student(students):
  roll = input("Enter Roll Number to delete: ").strip().upper()
  for s in students:
    if s["roll"] == roll:
      confirm = input(f"Are you sure you want to delete {s['name']}'s record? (yes/no):
").strip().lower()
      if confirm == "yes":
        students.remove(s)
        save_data(students)
        print("Student record deleted successfully.")
      else:
        print("Deletion cancelled.")
      return
  print("Student record not found.")

# Main Menu Control Loop
def main():
  students = load_data()
  while True:
    print("\n===== STUDENT PLACEMENT TRACKING SYSTEM =====")
    print("1. Add Student Profile")
    print("2. View All Records")
    print("3. Check Drive Eligibility")
    print("4. Update Placement Status")
```

```
    print("5. Delete Student Record")
    print("6. Exit")

    choice = input("Enter your choice (1-6): ").strip()
    if choice == "1":
      add_student(students)
    elif choice == "2":
      view_students(students)
    elif choice == "3":
      check_eligibility(students)
    elif choice == "4":
      update_placement(students)
    elif choice == "5":
      delete_student(students)
    elif choice == "6":
      print("Thank you for using the Placement Tracking System!")
      break
    else:
      print("Invalid choice. Please select between 1 and 6.")

if __name__ == "__main__":
  main()
```

OUTPUT:

Output 1: Adding a student and displaying the result sheet

```
===== STUDENT PLACEMENT TRACKING SYSTEM =====
1. Add Student Profile
2. View All Records
3. Check Drive Eligibility
4. Update Placement Status
5. Delete Student Record
6. Exit
Enter your choice (1-6): 1

--- Add Student Profile ---
Enter Roll Number: 101
Enter Student Name: Priya
Available Branches: CSE, ECE, EEE, IT, MECH, CIVIL
Enter Branch: CSE
Enter CGPA (0.0 - 10.0): 9.5
Enter Active Backlogs: 0
Student profile added successfully!

==========================================
         STUDENT PLACEMENT PROFILE
==========================================
Roll Number : 101
Name        : Priya
Branch      : CSE
CGPA        : 9.50
Backlogs    : 0
Status      : Unplaced
Company     : N/A
Package     : N/A
==========================================
```

Output 2: Input validation, a FAIL result, and a duplicate roll number

```
===== STUDENT PLACEMENT TRACKING SYSTEM =====
1. Add Student Profile
2. View All Records
3. Check Drive Eligibility
4. Update Placement Status
5. Delete Student Record
6. Exit
Enter your choice (1-6): 1

--- Add Student Profile ---
Enter Roll Number: 102
Enter Student Name: Ryan
Available Branches: CSE, ECE, EEE, IT, MECH, CIVIL
Enter Branch: CSE
Enter CGPA (0.0 - 10.0): -4.2
Enter a value between 0.0 and 10.0.
Enter CGPA (0.0 - 10.0): 8.5
Enter Active Backlogs: -45
Enter a value greater than or equal to 0.
Enter Active Backlogs: 3
Student profile added successfully!

==============================================
         STUDENT PLACEMENT PROFILE
==============================================
Roll Number : 102
Name        : Ryan
Branch      : CSE
CGPA        : 8.50
Backlogs    : 3
Status      : Unplaced
Company     : N/A
Package     : N/A
==============================================
```

Output 3: Viewing all students and searching by roll number

```
==========================================
         STUDENT PLACEMENT PROFILE
==========================================
Roll Number : 102
Name        : Ryan
Branch      : CSE
CGPA        : 8.50
Backlogs    : 3
Status      : Unplaced
Company     : N/A
Package     : N/A
==========================================

===== STUDENT PLACEMENT TRACKING SYSTEM =====
1. Add Student Profile
2. View All Records
3. Check Drive Eligibility
4. Update Placement Status
5. Delete Student Record
6. Exit
Enter your choice (1-6): 2

--- All Student Placement Records ---
Roll        Name             Branch  CGPA    Status      Company
-----------------------------------------------------------------------
101         Priya            CSE     9.50    Unplaced    N/A
102         Ryan             CSE     8.50    Unplaced    N/A
```

Output 4: Updating marks and deleting a record

```
===== STUDENT PLACEMENT TRACKING SYSTEM =====
1. Add Student Profile
2. View All Records
3. Check Drive Eligibility
4. Update Placement Status
5. Delete Student Record
6. Exit
Enter your choice (1-6): 4
Enter Roll Number to update placement status: 101
Current Status for Priya: Unplaced
Enter Placed Company Name: TCS
Enter Offered Package (e.g., 6.5 LPA): 6 LPA
Placement status updated successfully!

==========================================
         STUDENT PLACEMENT PROFILE
==========================================
Roll Number : 101
Name        : Priya
Branch      : CSE
CGPA        : 9.50
Backlogs    : 0
Status      : Placed
Company     : TCS
Package     : 6 LPA
==========================================
```

Time Complexity:

Operation	Complexity	Reason
Operation	**Time Complexity**	**Explanation**
Add Student Profile	$O(n)$	Scans all n existing records to check for duplicate roll numbers; appending to the list itself is $O(1)$.
View All Records	$O(n)$	Iterates through the list of all n student records once to display them in a tabular format.
Check Drive Eligibility	$O(n)$	Uses a list comprehension to scan and filter all n student records against the CGPA and backlog criteria.
Update Placement Status	$O(n)$	Performs a linear search by roll number through n records, followed by a direct update.
Delete Student Record	$O(n)$	Performs a linear search by roll number, followed by an $O(n)$ removal operation from the list.

Summary of Key Results:

The project efficiently stores, calculates, and displays student result data based on user input. Invalid marks, empty fields, and duplicate roll numbers are rejected, and the records are preserved in students.json between runs.

Interpretation:

Automatic calculation removes manual errors, and structured storage makes records quick to find, update, and display, making the system user-friendly.

Achievement of Objectives:

- Student marks management system implemented successfully.
- Grading algorithm applied effectively.
- Menu-driven interface developed.
- Input validation and file storage implemented.

Implications:

This system can be extended to real-world applications such as college examination systems, school result portals, and online learning platforms.

6 Mapping POs and SDGs

6.1 Program Outcomes (POs) Mapping

Table 1: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied data structures (list, dictionary, tuple) and a grading algorithm
PO2	Problem Analysis	Solved the problem of organising and evaluating student marks
PO3	Design/Development of Solutions	Developed a menu-driven student marks system
PO4	Conduct Investigations of Complex Problems	Tested the system with valid, invalid, and boundary inputs
PO5	Engineering Tool Usage	Used Python and the json and os modules

6.2 Sustainable Development Goals (SDGs) Mapping

Table 2: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
SDG 4	Quality Education	Helps institutions track student performance and helps students learn data structures
SDG 8	Decent Work and Economic Growth	Facilitates transparent and efficient campus placement drives.
SDG 12	Responsible Consumption and Production	Reduces paper-based record keeping

7 Conclusion

The **Student Placement Tracking System** successfully demonstrates the practical implementation of Python programming concepts, data structures, and file handling. By automating student eligibility checks and placement updates, the system ensures precision, eliminates manual errors, and provides an efficient tool for academic placement cells.

Future Work:

- Add sorting algorithms (such as Merge Sort or Quick Sort) to rank students and list toppers
- Use a dictionary keyed by roll number for O(1) searching
- Use database integration (SQLite/MySQL)
- Implement a GUI or web interface
- Add class statistics such as average, highest, and lowest marks

Bibliography

- [1] Python Software Foundation, *Python 3 Documentation*, Available: https://docs.python.org/3/. [Accessed: Oct. 2026].
- [2] GeeksforGeeks, *Python Data Structures and File Handling*, Available: https://www.geeksforgeeks.org/python-data-structures/. [Accessed: Oct. 2026].
- [3] TutorialsPoint, *Data Structures and Algorithms in Python*, Available: https://www.tutorialspoint.com/python_data_structures/index.htm. [Accessed: Oct. 2026].
- [4] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ: Prentice-Hall, 1988 (referenced for foundational procedural programming and modular structure principles).